Compositional reasoning for Vision

Ashwin Vinod, Minchao huang

Why Visual Reasoning Needs Structure

Modern vision-language models (e.g. CLIP, ViLT, Flamingo) are strong *pattern recognizers* but weak *reasoners*.

- More Data
- More Parameters
- More compute

❌ End-to-end models fail or are just compute intensive.

We need compositional, interpretable reasoning beyond end-to-end networks.

Why Visual Reasoning Needs Structure

Modern vision-language models (e.g. CLIP, ViLT, Flamingo) are strong *pattern recognizers* but weak *reasoners*.

- More Data
- More Parameters
- More compute

❌ End-to-end models fail or are just compute intensive.

Q) Tag the 7 main characters on the TV show Big Bang Theory

1. Detect Faces
2. Query knowledge
3. Match faces to names
4. Display bounding boxes

We need compositional, interpretable reasoning beyond end-to-end networks.

VISPROG Visual Programming for Compositional Visual Reasoning

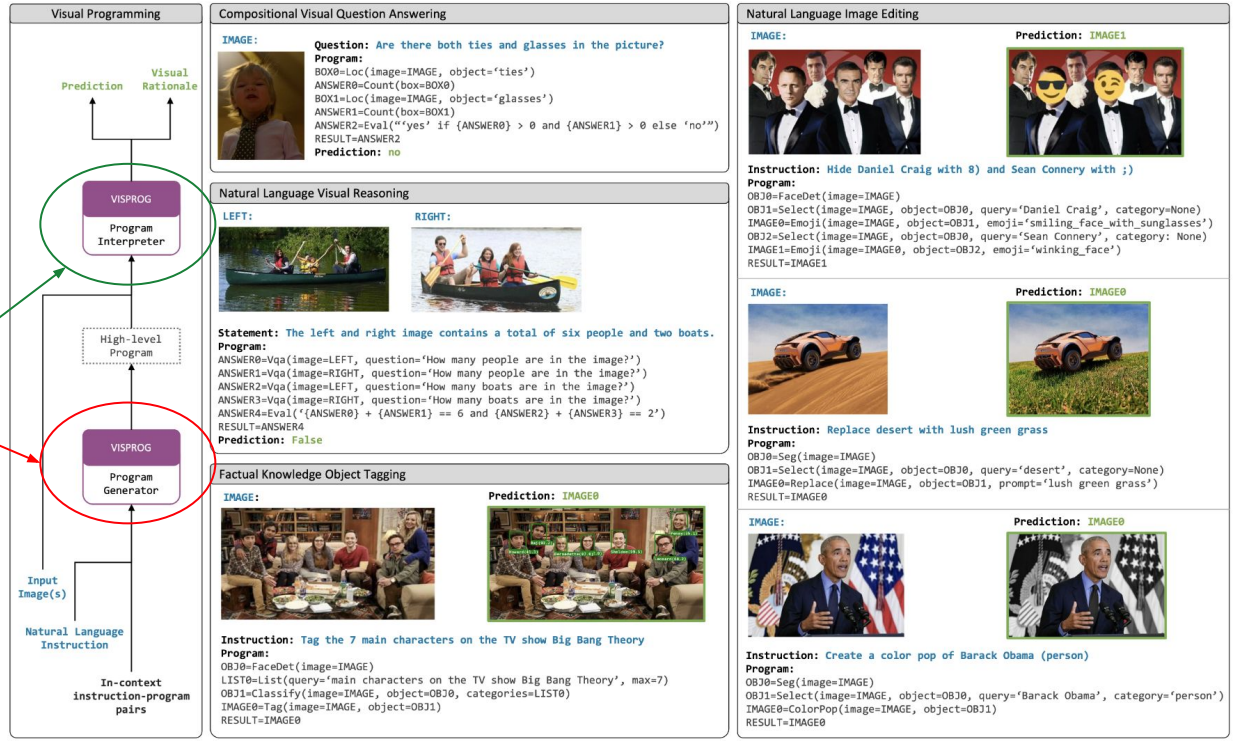
- A neuro-symbolic system for visual reasoning.
- It uses GPT-3's in-context learning to translate natural language instructions into Python-like programs

Gupta, Tanmay, and Aniruddha Kembhavi. "Visual programming: Compositional visual reasoning without training." Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2023.

VISPROG Visual Programming for Compositional Visual Reasoning

- A neuro-symbolic system for visual reasoning.
- It uses GPT-3's in-context learning to translate natural language instructions into Python-like programs

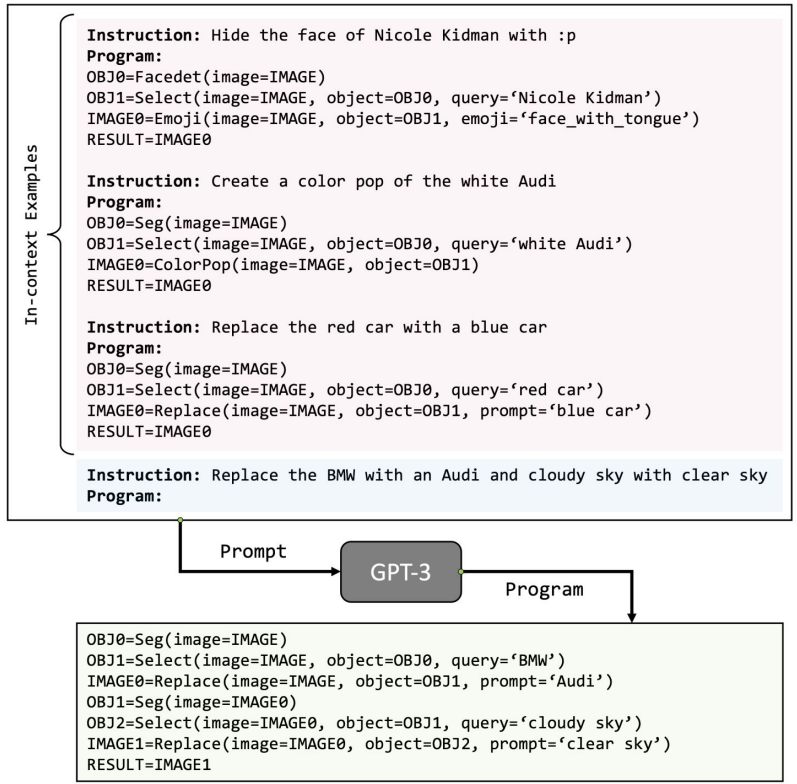
- The program is then executed step-by-step, producing both an answer and a visual rationale.



VISPROG Visual Programming for Compositional Visual Reasoning

- A neuro-symbolic system for visual reasoning.
- It uses GPT-3's in-context learning to translate natural language instructions into Python-like programs

- The program is then executed step-by-step, producing both an answer and a visual rationale.

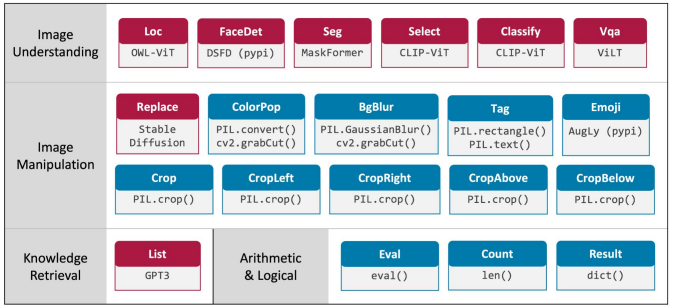


VISPROG Architecture

Input: Instruction + image(s).

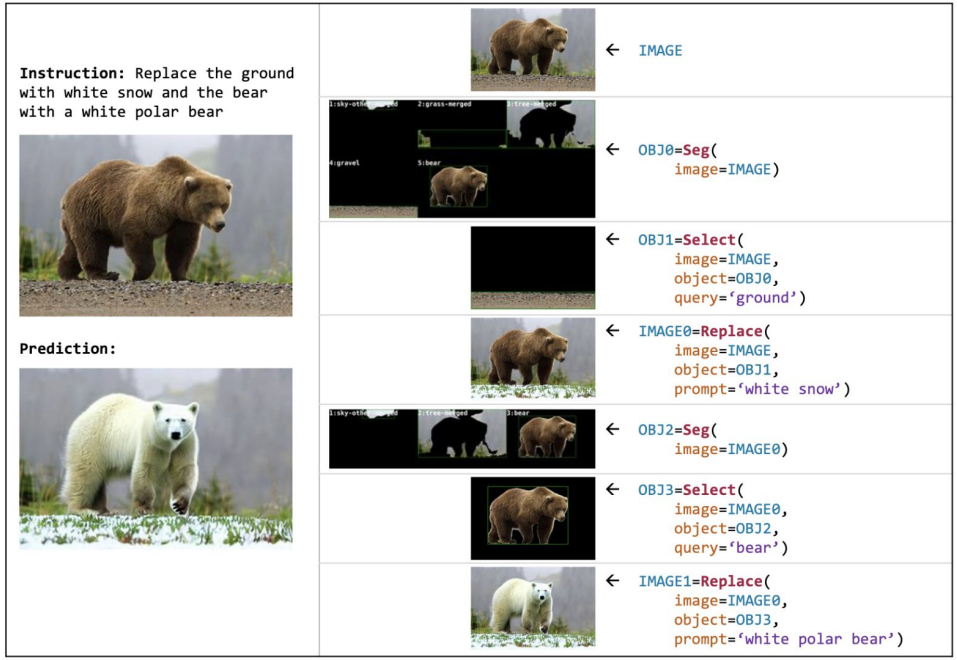
- Step 1: GPT-3 generates a program (sequence of module calls).
- Step 2: VISPROG interpreter executes each module and updates a shared state.
- Step 3: Intermediate outputs are composed into a visual rationale (HTML summary). 20 modules across perception, manipulation, logic.

```
class VisProgModule():
  def __init__(self):
    # load a trained model; move to GPU

  def html(self,inputs: List,output: Any):
    # return an html string visualizing step I/O

  def parse(self,step: str):
    # parse step and return list of input values
    # and variables, and output variable name

  def execute(self,step: str,state: Dict):
    inputs, input_var_names, output_var_name = \
      self.parse(step)

    # get values of input variables from state
    for var_name in input_var_names:
      inputs.append(state[var_name])

    # perform computation using the loaded model
    output = some_computation(inputs)

    # update state
    state[output_var_name] = output

    # visual summary of the step computation
    step_html = self.html(inputs,output)
    return output, step_html
```



⭐ Easy extensibility [add a new module = add a new tool.]

Visual Rationales in Action



Every step's input/output is visible → users can verify reasoning logic.

What can VISPROG do?

- **Compositional VQA (GQA[1])** – + 2.7 pts Acc. vs **ViLT** .
- **Image-Pair Reasoning (NLVRv2)** – 62.4 % zero-shot.
- **Natural Language Image Editing** – uses Stable Diffusion for semantic edits.
- **No fine-tuning** – just in context demonstrations.

Task	Input	Output	Modules
Compositional Visual QA (GQA)	Image + Question	Text	Loc, Vqa, Eval, Count, Crop, CropLeft, CropRight, CropAbove, CropBelow
Reasoning on Image Pairs (NLVR)	Image Pair + Statement	True/False	Vqa, Eval
Factual Knowledge Object Tagging	Image + Instruction	Image	FaceDet, List, Classify, Loc, Tag
Image Editing with Natural Language	Image + Instruction	Image	FaceDet, Seg, Select, Replace, ColorPop, BgBlur, Emoji

[1] https://arxiv.org/abs/1902.09506 [2] https://yoavartzi.com/pub/slya-acl.2017.pdf

What can VISPROG do?

Compositional Visual Question Answering (GQA)	Reasoning on Image Pairs (NLVR²)	Factual Knowledge Object Tagging	Language-Guided Image Editing
• GQA dataset on Accuracy • 50.5 % (voting) +2.7 points over ViLT VQA baseline	• NLVR² dataset Accuracy • 62.4% zero-shot without fine tuning on NLVR²	• 100 instructions/ Custom (46 images) on Precision / Recall / F1 (%) • Tagging F1 = 63.7 → 75.7 (after instruction tuning) • Localization F1 = 80.6% → 84.9% (after instruction tuning)	• 107 instructions/ Custom (65 images) • semantic correctness of the edit measured manually • 59.8 % → 66.4 % (after instruction tuning)

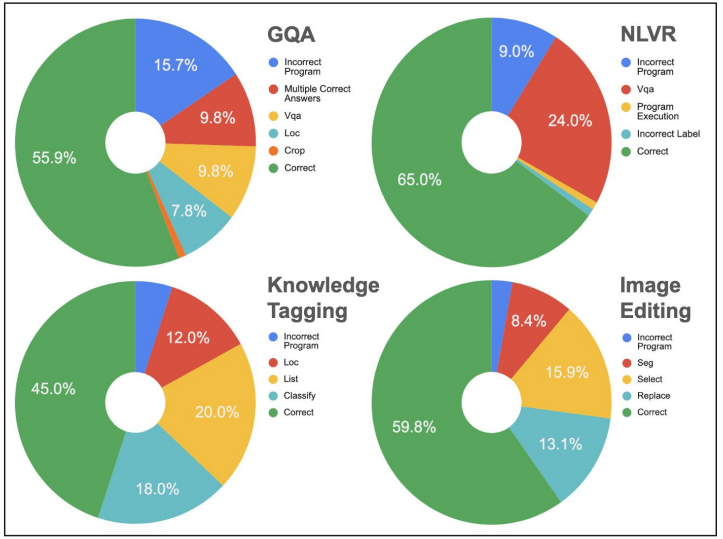
Interpretable, Modular, but Prompt-Dependent

✅ Strengths

- Transparent reasoning (visual rationales).
- Modular and easily extensible (20+ modules).
- Zero-training = adaptable to new tasks quickly.

⚠️ Limitations

- Dependent on prompt examples (manual in-context programming).
- No feedback or self-correction loop.
- Latency: multiple model calls per program.

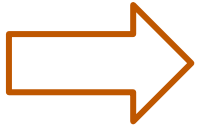


Sources of error analysis on 100 samples for each task

From Visual Programs to Executable Reasoning

VISPROG

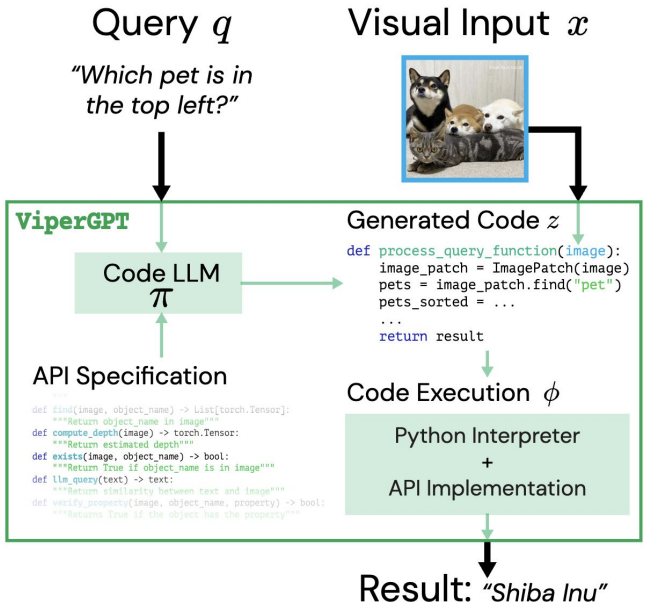
- Custom interpreter (limited control flow)
- Required hand-crafted in-context examples



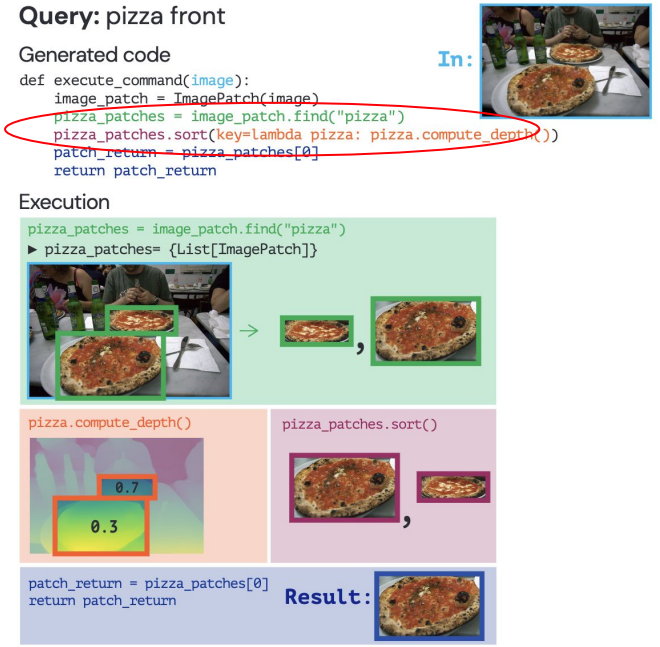
ViperGPT

- Writes real Python code and executes directly using Python interpreter
- Does not use incontext examples but rather function definitions.

Overview



Sample input /output



References: Surís, Dídac, Sachit Menon, and Carl Vondrick. "Vipergpt: Visual inference via python execution for reasoning." Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023.

API Specification

Specifies:

- Input and output types
- Docstrings
- Examples

❗ Only specifications, no full implementation

- Limited context windows
- Code generation is independent of the changes made to the module implementation

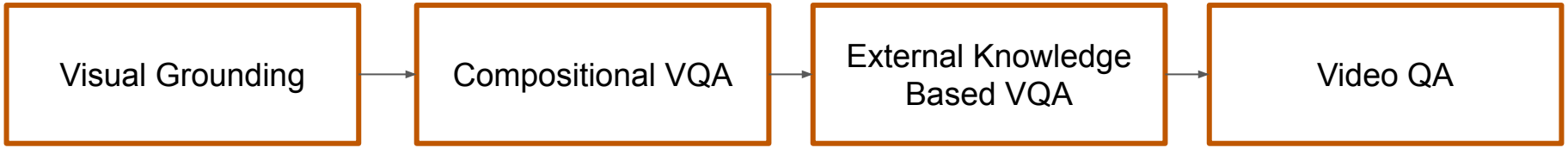
```
def exists(self, object_name: str) -> bool:
    """Returns True if the object specified by object_name is found in the image, and False otherwise.
    Parameters
    -------
    object_name : str
        A string describing the name of the object to be found in the image.

    Examples
    -------
    >>> # Are there both cakes and gummy bears in the photo?
    >>> def execute_command(image)->str:
    >>>     image_patch = ImagePatch(image)
    >>>     is_cake = image_patch.exists("cake")
    >>>     is_gummy_bear = image_patch.exists("gummy bear")
    >>>     return bool_to_yesno(is_cake and is_gummy_bear)
    """
    return len(self.find(object_name)) > 0
```

```
def llm_query(question: str) -> str:
    '''Answers a text question using GPT-3. The input question is always a formatted string with a variable in it.

    Parameters
    ----------
    question: str
        the text question to ask. Must not contain any reference to 'the image' or 'the photo', etc.
    '''
    return llm_query(question)
```

References: Surís, Dídac, Sachit Menon, and Carl Vondrick. "Vipergpt: Visual inference via python execution for reasoning." Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023.

Evaluation on Tasks

- Defines 4 main tasks ranging from basic understanding to complex synthesis
- Each task is a "prerequisite" for following task



- RefCOCO and RefCOCO+ on IoU
- 72.0 / 67.0 +13-17 points ↑ over strongest zero-shot baselines(OWL-ViT, GLIPetc)

- GQA* dataset on Accuracy
- 48.1 % +3.4 points ↑ over BLIP-2.

- OK-VQA dataset on Accuracy
-
- 51.9 % +1.3 points ↑ vs Flamingo; +16 points ↑ vs PNP-VQA.
- First to match SOTA zero-shot knowledge VQA.

- NExT-QA dataset on Accuracy
- 49.8 (T) / 56.4 (C) / 60.0
- Matches or exceeds supervised models.

Query: pizza front

Generated code

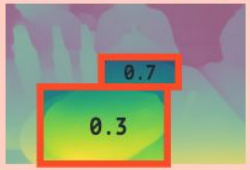
In:

```
def execute_command(image):
    image_patch = ImagePatch(image)
    pizza_patches = image_patch.find("pizza")
    pizza_patches.sort(key=lambda pizza: pizza.compute_depth())
    patch_return = pizza_patches[0]
    return patch_return
```

Execution

```
pizza_patches = image_patch.find("pizza")
▶ pizza_patches= {List[ImagePatch]}
```

pizza.compute_depth()



pizza_patches.sort()

```
patch_return = pizza_patches[0]
return patch_return
```

Result:

sis

xternal Knowledge Based VQA

Video QA

OK-VQA dataset on Accuracy

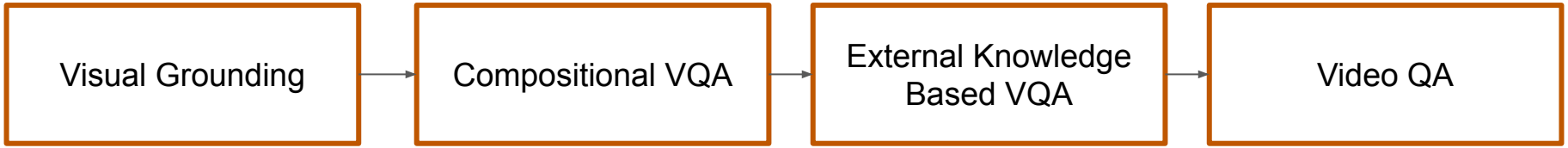
Better than zero-shot and on-par with some fine-tuned models

- NExT-QA dataset on Accuracy
- Achieves SoTA results, surpassing fine-tuned methods
- frames relations for temporal and causal.

Methods.

Evaluation on Tasks

- Defines 4 main tasks ranging from basic understanding to complex synthesis
- Each task is a "prerequisite" for following task



- RefCOCO and RefCOCO+ on IoU
- 72.0 / 67.0 +13-17 points ↑ over strongest zero-shot baselines(OWL-ViT, GLIPetc)

- GQA* dataset on Accuracy
- 48.1 % +3.4 points ↑ over BLIP-2.

- OK-VQA dataset on Accuracy
-
- 51.9 % +1.3 points ↑ vs Flamingo; +16 points ↑ vs PNP-VQA.
- First to match SOTA zero-shot knowledge VQA.

- NExT-QA dataset on Accuracy
- 49.8 (T) / 56.4 (C) / 60.0
- Matches or exceeds supervised models.

Evaluation on Tasks

- Defines 4 main tasks ranging from basic understanding to complex synthesis
- Each task is a "prerequisite" for following task

Query: Are there water bottles to the right of the bookcase that is made of wood?

In:

Generated code

```
def execute_command(image):
    image_patch = ImagePatch(image)
    bookcase_patches = image_patch.find("bookcase")
    for bookcase_patch in bookcase_patches:
        is_wood = bookcase_patch.verify_property("bookcase", "wood")
        if is_wood:
            water_bottle_patches = image_patch.find("water bottle")
            for water_bottle_patch in water_bottle_patches:
                if water_bottle_patch.horizontal_center > \
                   bookcase_patch.horizontal_center:
                    return "yes"
            return "no"
    return "no"
```

Execution

```
bookcase_patches= image_patch.
      find("bookcase")
▶ bookcase_patches[0] = {ImagePatch}
```

```
▶ bookcase_patches[0].
     horizontal_center = {float} 239.0
...verify_property("bookcase","wood")
▶ is_wood = {bool} True
```

```
water_bottle_patches = image_patch.
      find("water bottle")
▶ water_bottle_patches[0]
   = {ImagePatches}
```

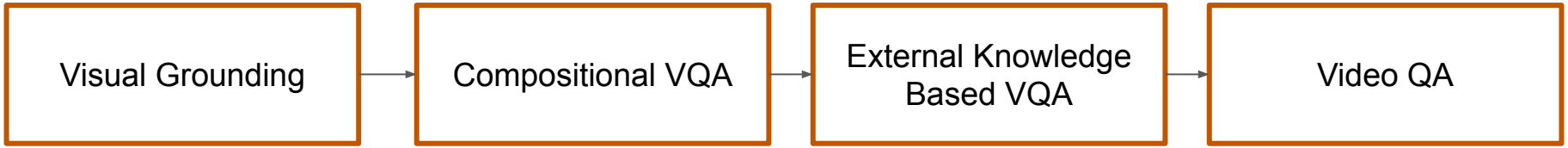
```
▶ water_bottle_patches[0].
     horizontal_center = {float} 608.5
▶ water_bottle_patch.horizontal_center >
bookcase_patch.horizontal_center =
{bool} True
```

Result: "yes"

- Still far behind fine-tuned models
- Decently far behind all fine-tuned Methods.
- models
- frames relations for temporal and causal.

Evaluation on Tasks

- Defines 4 main tasks ranging from basic understanding to complex synthesis
- Each task is a "prerequisite" for following task



- RefCOCO and RefCOCO+ on IoU
- 72.0 / 67.0 +13-17 points ↑ over strongest zero-shot baselines(OWL-ViT, GLIPetc)

- GQA* dataset on Accuracy
- 48.1 % +3.4 points ↑ over BLIP-2.

- OK-VQA dataset on Accuracy
-
- 51.9 % +1.3 points ↑ vs Flamingo; +16 points ↑ vs PNP-VQA.
- First to match SOTA zero-shot knowledge VQA.

- NExT-QA dataset on Accuracy
- 49.8%(T) / 56.4% (C) / 60.0%
- Matches or exceeds supervised models.(ATP,HiTeA)

Evaluation on Tasks

Query: What did the boy do after he dropped the sparkles on the floor?

Generated code

```
def execute_command(video, question, possible_answers):
    video_segment = VideoSegment(video)
    drop_detected = False
    for i, frame in enumerate(video_segment.frame_iterator()):
        if frame.exists("boy") and frame.exists("sparkles") and \
                frame.simple_query("is the boy dropping the sparkles?") == "yes":
            drop_detected = True
            break
    if drop_detected:
        index_frame = i + 1
    else:
        index_frame = video_segment.num_frames // 2
    frame_of_interest = ImagePatch(video_segment, index_frame)
    boys = frame_of_interest.find("boy")
    if len(boys) == 0:
        boys = [frame_of_interest]
    boy = boys[0]
    caption = boy.simple_query("What is this?")
    info = {
        "Caption of frame after dropping the sparkles": caption,
    }
    answer = select_answer(info, question, possible_answers)
    return answer
```

Execution

In:

```
frame.exists("boy") and \
frame.exists("sparkles") and \
frame.simple_query("is the boy
   dropping the sparkles?") == "yes":
```

▶ frame = {ImagePatch}

▶ i= {int} 25

```
index_frame = i + 1
```

▶ index_frame = {int} 26

▶ frame_of_interest = {ImagePatch}

```
boys = frame_of_interest.find("boy")
```

▶ boy = {ImagePatch}

▶ caption = {str} "a child running with fire in his hands"

▶ answer = {str} "pick it up"

Result: *"Pick it up"*

fine-tuned models

all fine-tuned Methods.

- frames relations for temporal and causal.

Emergent Capabilities

Query: What would the founder of the brand of the car on the left say to the founder of the brand of the car on the right?


```
def execute_command(image):
    image_patch = ImagePatch(image)
    car_patches = image_patch.find("car")
    car_patches.sort(key=lambda car: car.horizontal_center)
    left_car = car_patches[0]
    right_car = car_patches[-1]
    left_car_brand = left_car.simple_query("What is the brand of this car?")
    right_car_brand = right_car.simple_query("What is the brand of this car?")
    left_car_founder = llm_query(f"Who is the founder of {left_car_brand}?")
    right_car_founder = llm_query(f"Who is the founder of {right_car_brand}?")
    return llm_query(f"What would {left_car_founder} say to {right_car_founder}?")
```

```
car_patches =
image_patch.find("car")
```

```
car_patches.sort(...)
```

```
▶left_car_brand='lamborghini'
▶right_car_brand='ferrari'
```

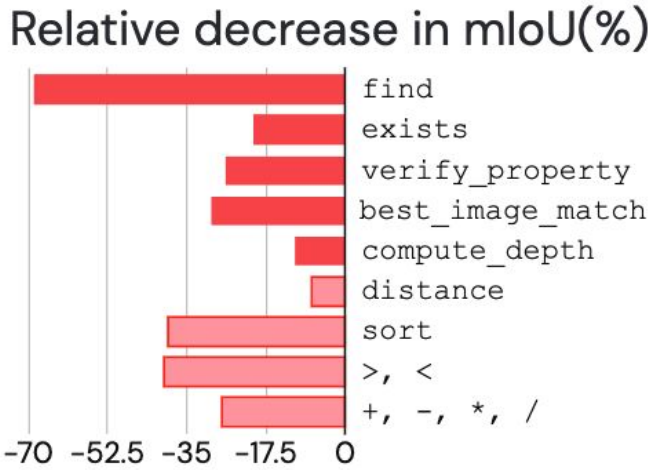
```
▶left_car_founder='Ferruccio Lamborghini'
▶right_car_founder='Enzo Ferrari'
```

Result: *"Ferruccio Lamborghini might say, 'It's been an honor to be a rival of yours for so many years, Enzo. May our cars continue to push each other to be better and faster!' "*

Viper-GPT easily adapts to real-world queries

References: Surís, Dídac, Sachit Menon, and Carl Vondrick. "Vipergpt: Visual inference via python execution for reasoning." ICCV 2023

Intervention | Context-Aware Responses



Query: Return the car that is on the correct lane

```
# Context: the picture was taken in the US
def execute_command(image):
    cars = image.find("car")
    for car in cars:
        if car.horizontal_center > image.horizontal_center:
            return car
    return None
```

Result: *None*

```
# Context: the picture was taken in the UK
def execute_command(image):
    cars = image.find("car")
    for car in cars:
        if car.horizontal_center < image.horizontal_center:
            return car
    return None
```

Result:

References: Surís, Dídac, Sachit Menon, and Carl Vondrick. "Vipergpt: Visual inference via python execution for reasoning." ICCV 2023

Strengths and Weaknesses

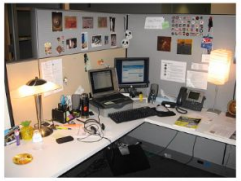
✅ Interpretability Through Explicit Programs
✅ Zero-shot, Task-agnostic Framework
✅ Compositional Reasoning
✅ Modular and Extensible

❌ Heavy Dependence on Code Generation Quality
❌ Reliance on Individual Module Performance
❌ Potentially High Computational and Latency Costs
❌ Lacks general instruction-following capabilities

How do we make these systems reliable?

- VISPROG & ViperGPT = great at modular reasoning, but brittle:
  - ~30% failures = LLM planning errors
  - ~40% = visual module errors (GQA Manual analysis)

Q: Is the keyboard to the left or to the right of the book?



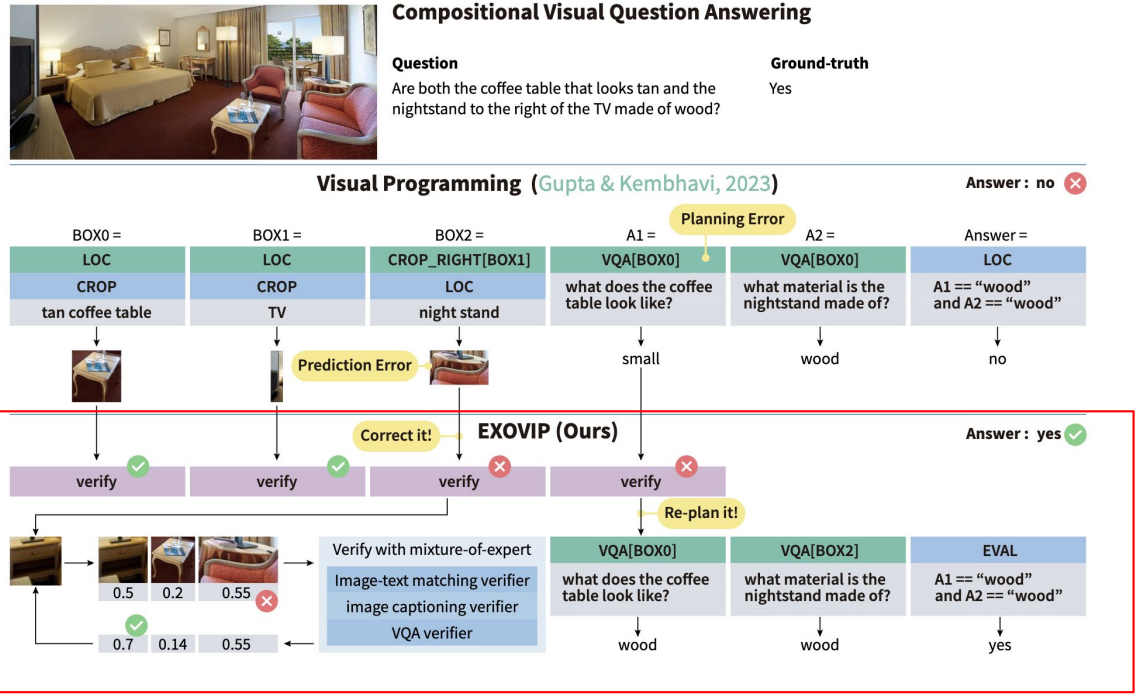
```
BOX0=LOC(image=IMG, object="book")
IMAGE0=CROP(bbox=BOX0)
BOX1=LOC(image= IMAGE0, object="keyboard")
# there is no keyboard in IMAGE0
```

Q: What's the woman holding?

```
BOX0=LOC(image=IMG, object="woman") = NONE
# no box is returned, but there should be one
```

How do we make these systems reliable?

- **ExoViP**[1] proposes a "plug-and-play exoskeleton" that verifies each reasoning step and corrects it.



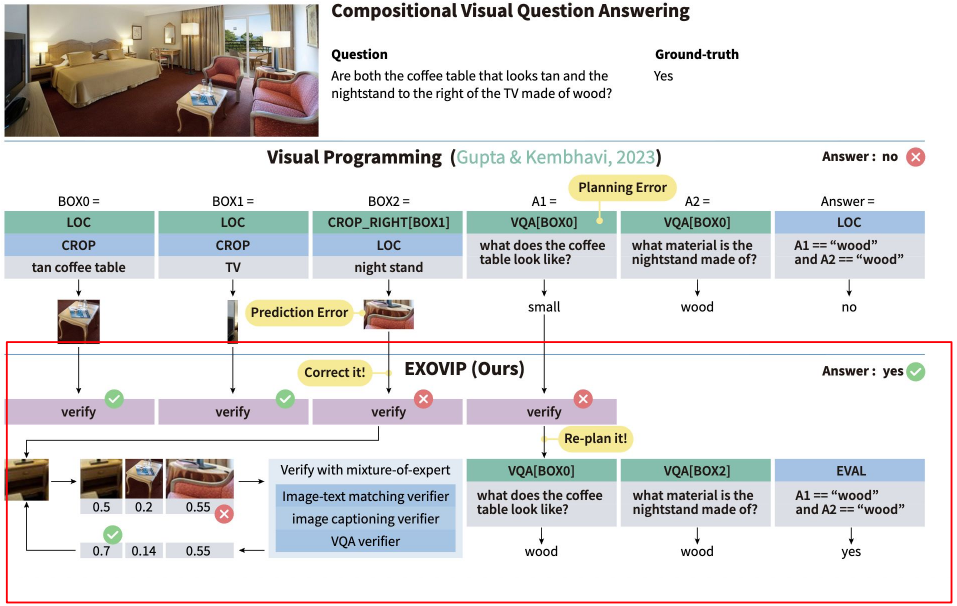
[1] Wang, Yuxuan, et al. "Exovip: Step-by-step verification and exploration with exoskeleton modules for compositional visual reasoning." arXiv preprint arXiv:2408.02210 (2024).

Verification as an Exoskeleton

ExoViP adds **verification modules** that “wrap around” each reasoning step.
These *exoskeletons* check correctness *after every step*, using three “sub-verifiers”:

- **Image-Text Matching Verifier (CLIP-based)** — checks if result visually fits description.
- **Image Captioning Verifier (BLIP)** — compares captioned scene to expectation.
- **VQA Verifier** — asks yes/no validation questions about each output.

Each produces a *verification score* → system adjusts or re-plans if inconsistency found.



[1] Wang, Yuxuan, et al. "Exovip: Step-by-step verification and exploration with exoskeleton modules for compositional visual reasoning." arXiv preprint arXiv:2408.02210 (2024).

Verification as an Exoskeleton

Initialize Tree:
Start with LLM-generated program; each child = next module step.

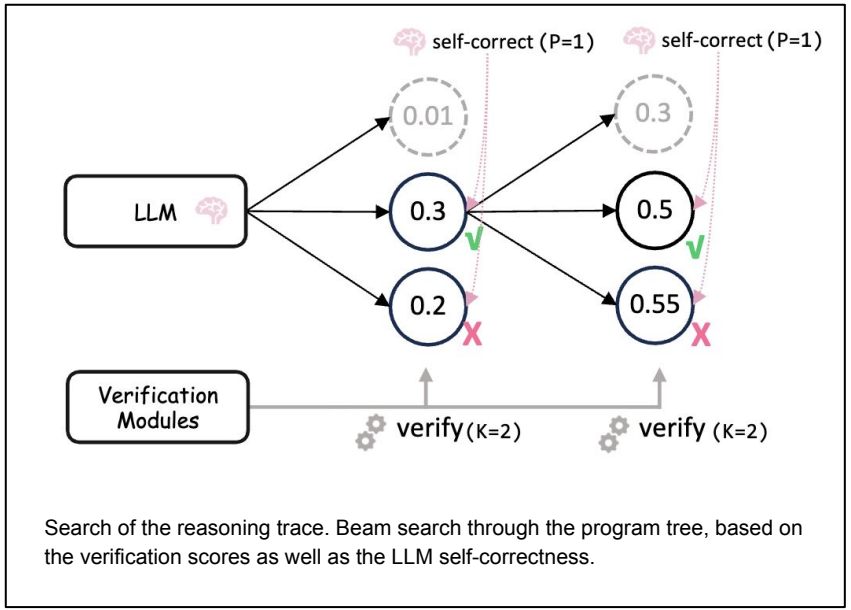
Execute & Verify:
Run each step → get verifier scores → normalize as weights.

Calibrate:
Combine module confidence & verifier weight → $p' = w \times p$.

Beam Search:
Keep top-K traces each expansion (branching ≈ 3).

Post-hoc Self-Correction (PSC):
If scores similar → LLM ranks traces → keep top-P (P < K).

Select Final Trace:
Choose path with highest combined verification + self-consistency score.



Search of the reasoning trace. Beam search through the program tree, based on the verification scores as well as the LLM self-correctness.

[1] Wang, Yuxuan, et al. "Exovip: Step-by-step verification and exploration with exoskeleton modules for compositional visual reasoning." arXiv preprint arXiv:2408.02210 (2024).

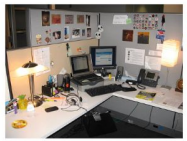
Mechanism: “Verify, Calibrate, Re-plan”

Q: What's the woman holding?

```
BOX0=LOC(image=IMG, object="woman") = NONE
# no box is returned, but there should be one
# after verification, this error is corrected
IMAGE0=CROP(bbox=BOX0)
ANSWER0=VQA(image=IMAGE0,
            question="what is the woman holding?")
```

(a) module error

Q: Is the keyboard to the left or to the right of the book?



```
BOX0=LOC(image=IMG, object="book")
IMAGE0=CROP(bbox=BOX0)
BOX1=LOC(image= IMAGE0, object="keyboard")
# there is no keyboard in IMAGE0
# after verification, the program is corrected:
# CROP ⟶ CROP_RIGHT
ANSWER0=COUNT(bbox= BOX1)
ANSWER1=EVAL(expression="'left'
             if ANSWER0 > 0  else 'right'")
```

(b) planning error

[1] Wang, Yuxuan, et al. "Exovip: Step-by-step verification and exploration with exoskeleton modules for compositional visual reasoning." arXiv preprint arXiv:2408.02210 (2024).

Results

Applied on **VISPROG** and **ViperGPT** pipelines → consistent gains:

- On **GQA** compositional VQA: from 57.4 → 61.5 accuracy (+4%).
- On **RefCOCO**: from 27.3 → 31.5 IoU (+4.2).
- Also improved abstract reasoning, image editing, and video reasoning tasks.
- Works as a universal add-on that improves both VISPROG and ViperGPT without retraining.

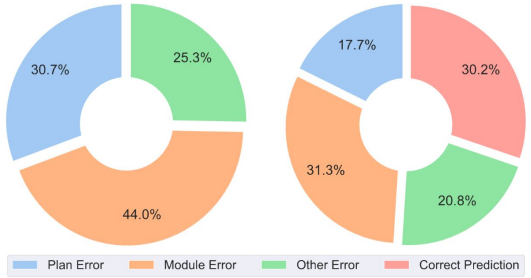
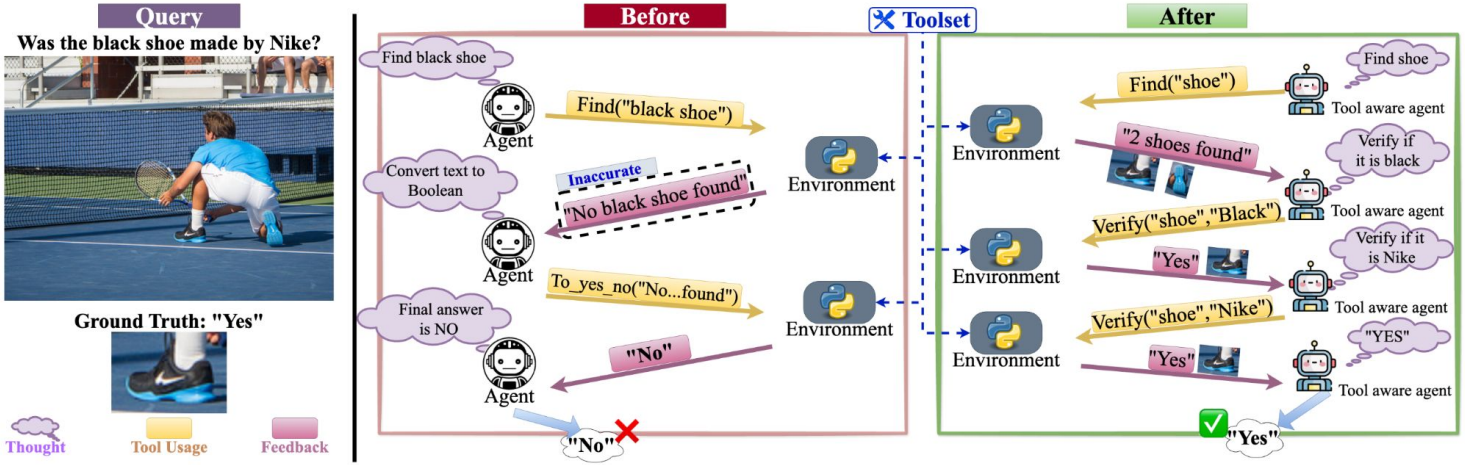


Figure 4: Distribution of the failure cases of original VISPROG (left), and distribution of the failure cases of EXOVIP (right)

DWIM (ICCV 2025): Teaching Agents to Use Tools Wisely

- Vis Prog, ViperGpt and Even self-correcting agents (ExoViP) don’t truly understand tool behavior.
- DWIM Train LLMs to become tool-aware, recognizing when tools fail and improving their use.

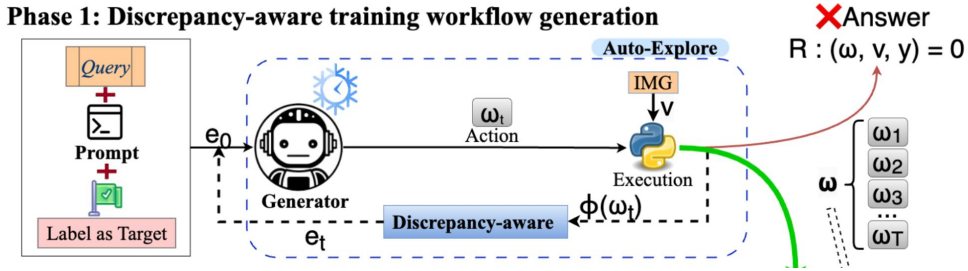


Discrepancy-Aware Workflow Generation

Standard compositional agents assume every tool output is correct.
DWIM adds “discrepancy detection”:

- Compares expected answer vs tool feedback.
- If mismatch → generates a “Rethink” action (natural-language note explaining tool failure + next plan).
- Collects these as training workflows that include both successes and failures.

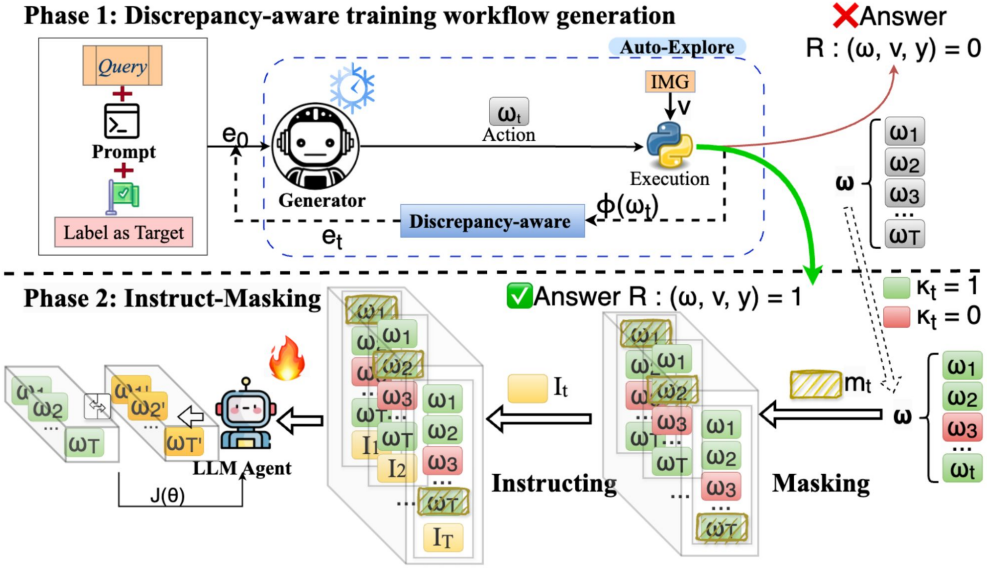
This creates richer supervision, not just right answers, but why wrong steps failed.



Instruct-Masking Fine-Tuning

DWIM trains the LLM to copy only effective actions.

- Each workflow step is labeled as effective or ineffective.
- During training, DWIM masks only the effective actions and instructs the model to regenerate them.
- This prevents the model from cloning noisy or incorrect tool usage.
- Essentially:
  - Naive SFT: copies everything (good + bad).
  - DWIM: copies only what worked — becoming tool-aware.
- Inspired by BERT's masking + instruction-tuning.



Results & Impact

Performance

- +9.7 % avg. improvement over HYDRA/VisRep.
- GQA 69.3 %, OK-VQA 62.8 %, SugarCREPE 74.6 %.

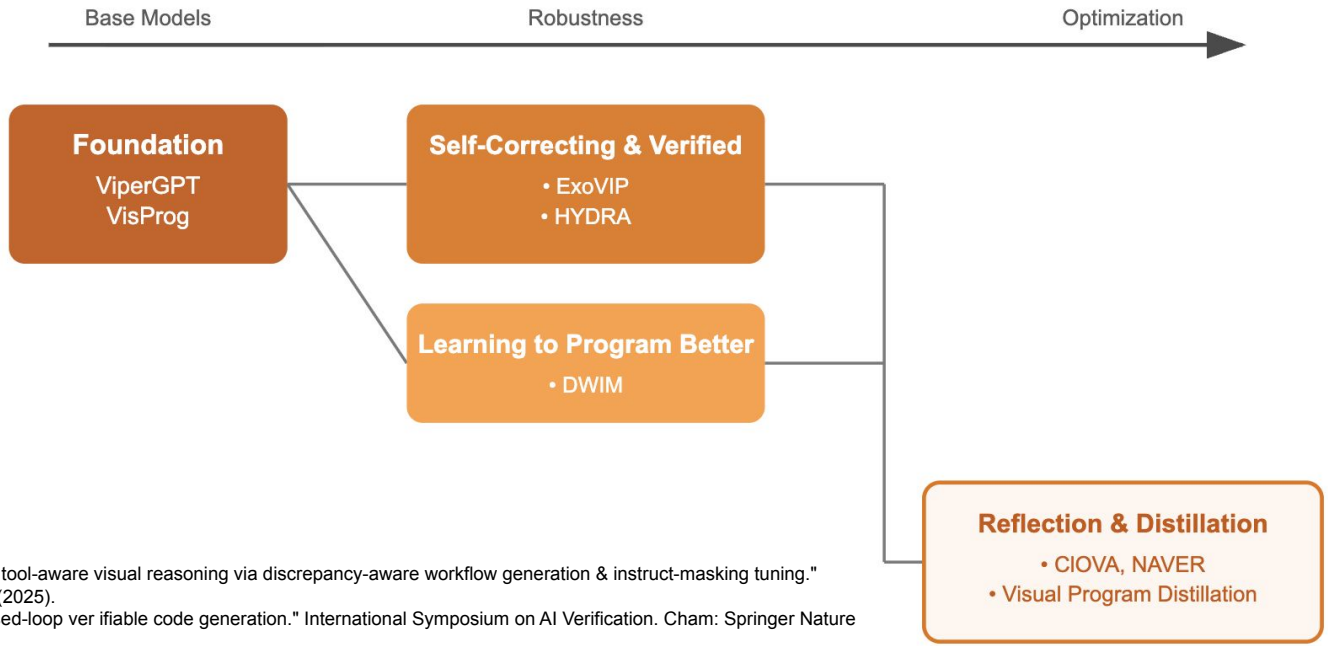
Generalization :

- Cross-dataset transfer → +4–7 % gain.

Tool efficiency:

- Average tool calls ↓ from ~3.3 (HYDRA) → 1.7 (DWIM).

From Programs to Agents and Learners



Ke, Fucai, et al. "Dwim: Towards tool-aware visual reasoning via discrepancy-aware workflow generation & instruct-masking tuning." arXiv preprint arXiv:2503.19263 (2025).
Sun, Chuyue, et al. "Clover: Closed-loop ver ifiable code generation." International Symposium on AI Verification. Cham: Springer Nature Switzerland, 2024.
Cai, Zhixi, et al. "Naver: A neuro-symbolic compositional automaton for visual grounding with explicit logic reasoning." arXiv preprint arXiv:2502.00372 (2025).
Hu, Yushi, et al. "Visual program distillation: Distilling tools and programmatic reasoning into vision-language models." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024.

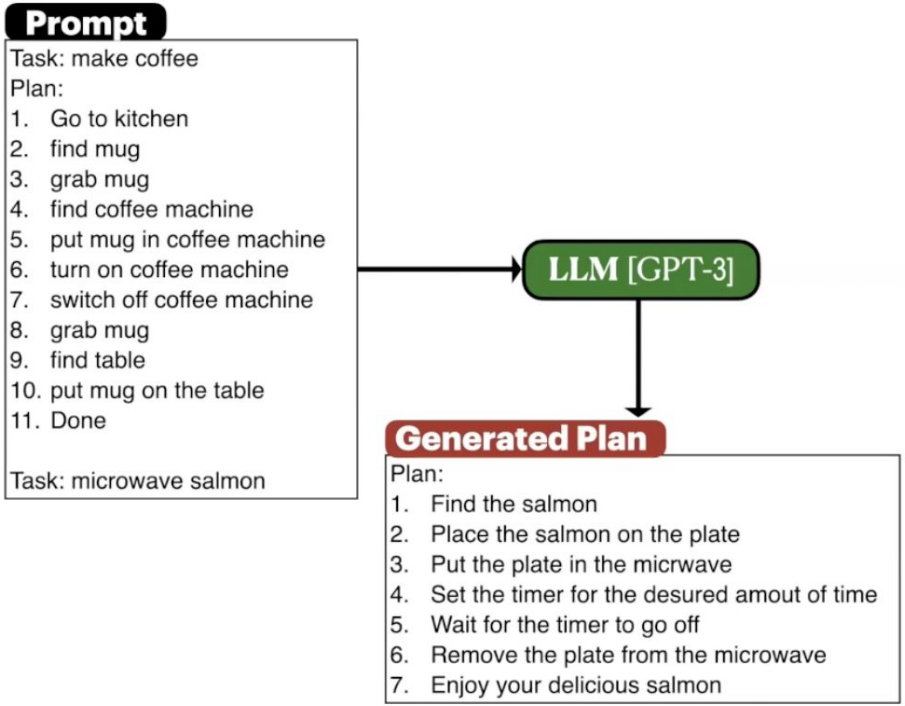
Questions?

PROGPROMPT: Generating Situated Robot Task Plans using Large Language Models

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, Animesh Garg

ICRA 2023

Background - LLMs for Task Planning

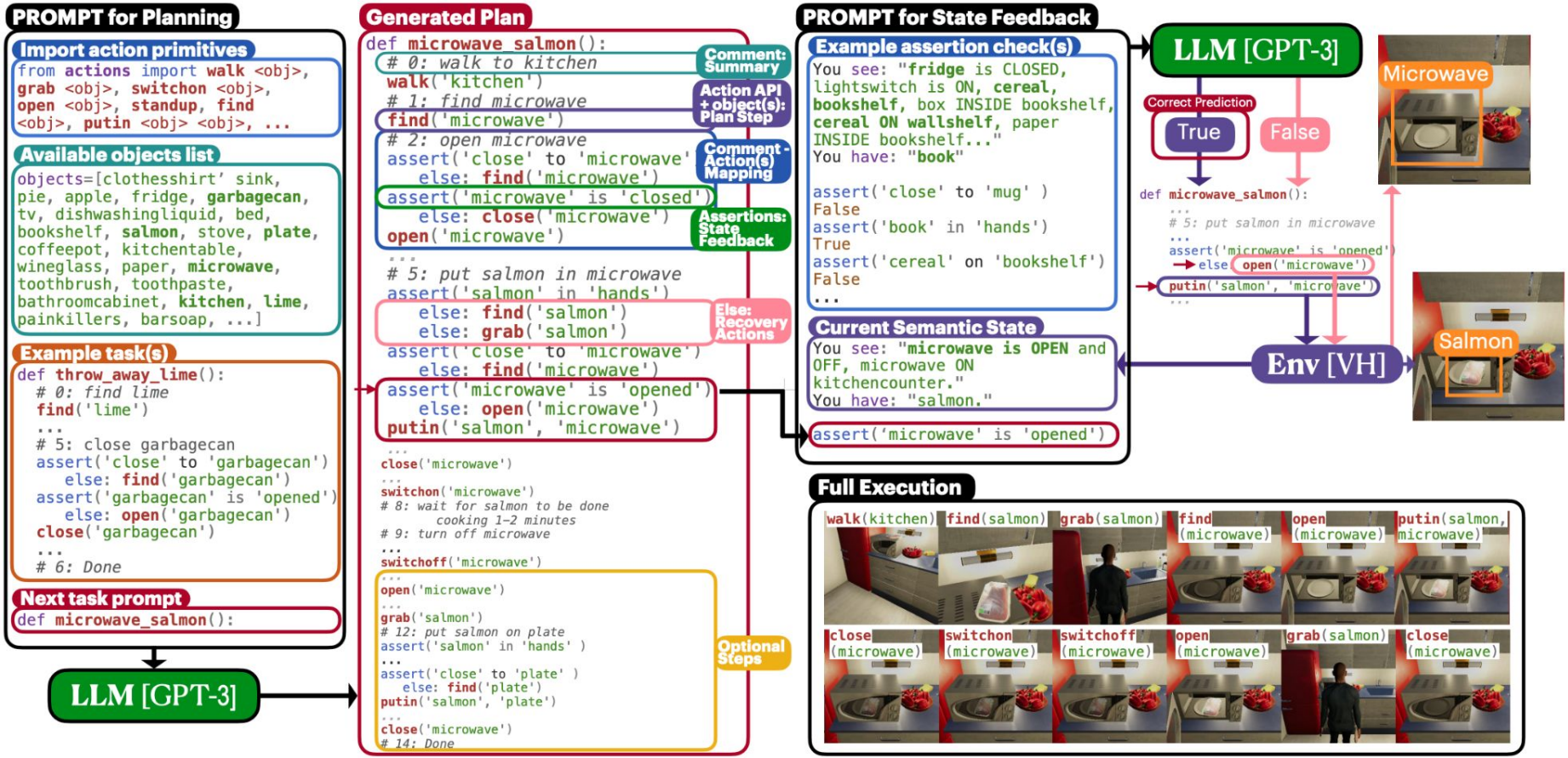


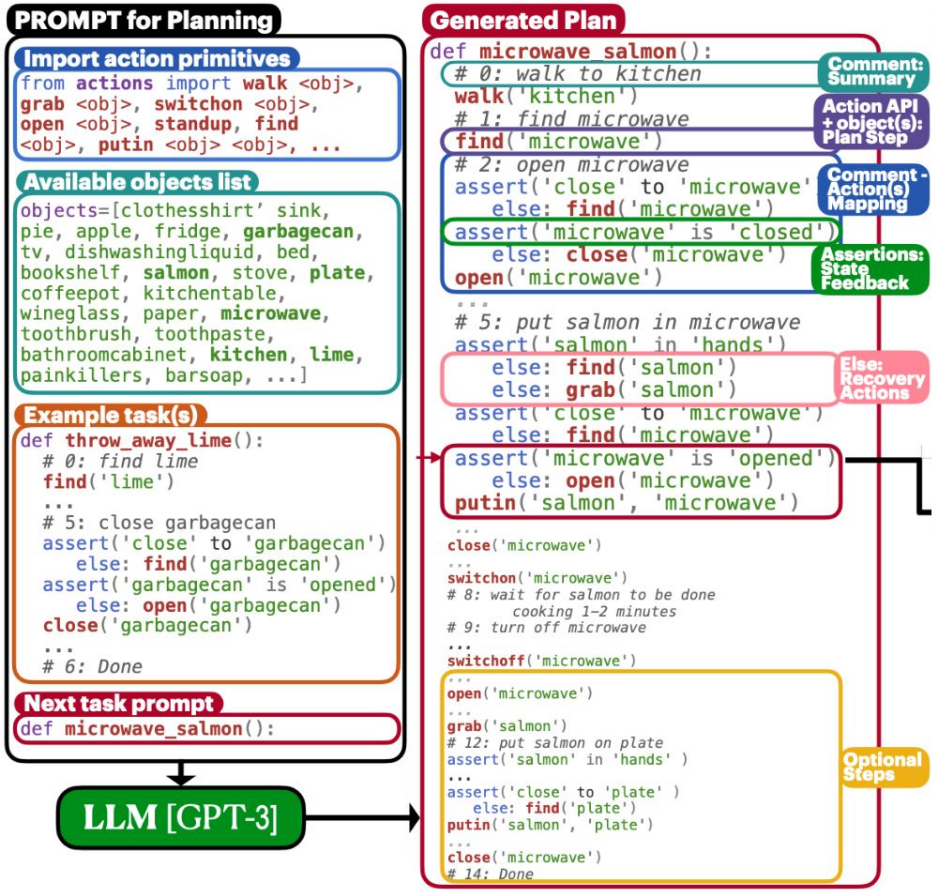
Challenges:

- LLM is not situated in the scene
- Text-to-robot action mapping may not be trivial
- Plan Steps using unavailable actions and objects
- Combinatorial admissible action space

Solution

Instead of using natural language, prompt the LLM with program-like specifications of the available actions and objects in an environment


PROMPT for Planning
Import action primitives
from actions import walk <obj>, grab <obj>, switchon <obj>, open <obj>, standup, find <obj>, putin <obj> <obj>, ...
Available objects list
objects=[clothesshirt' sink, pie, apple, fridge, garbagecan, tv, dishwashingliquid, bed, bookshelf, salmon, stove, plate, coffeepot, kitchentable, wineglass, paper, microwave, toothbrush, toothpaste, bathroomcabinet, kitchen, lime, painkillers, barsoap, ...]
Example task(s)
def throw_away_lime():
# 0: find lime
find('lime')
...
# 5: close garbagecan
assert('close' to 'garbagecan')
else: find('garbagecan')
assert('garbagecan' is 'opened')
else: open('garbagecan')
close('garbagecan')
...
6: Done
Next task prompt
def microwave_salmon():
LLM [GPT-3]
Generated Plan
def microwave_salmon():
# 0: walk to kitchen
walk('kitchen')
# 1: find microwave
find('microwave')
# 2: open microwave
assert('close' to 'microwave')
else: find('microwave')
assert('microwave' is 'closed')
else: close('microwave')
open('microwave')
...
# 5: put salmon in microwave
assert('salmon' in 'hands')
else: find('salmon')
else: grab('salmon')
assert('close' to 'microwave')
else: find('microwave')
assert('microwave' is 'opened')
else: open('microwave')
putin('salmon', 'microwave')
...
close('microwave')
...
switchon('microwave')
# 8: wait for salmon to be done
cooking 1-2 minutes
9: turn off microwave
...
switchoff('microwave')
...
open('microwave')
...
grab('salmon')
# 12: put salmon on plate
assert('salmon' in 'hands')
...
assert('close' to 'plate')
else: find('plate')
putin('salmon', 'plate')
...
close('microwave')
# 14: Done
Comment: Summary
Action API + object(s): Plan Step
Comment - Action(s) Mapping
Assertions: State Feedback
Else: Recovery Actions
Optional Steps
PROMPT for State Feedback
Example assertion check(s)
You see: "fridge is CLOSED, lightswitch is ON, cereal, bookshelf, box INSIDE bookshelf, cereal ON wallshelf, paper INSIDE bookshelf..."
You have: "book"
assert('close' to 'mug')
False
assert('book' in 'hands')
True
assert('cereal' on 'bookshelf')
False
...
Current Semantic State
You see: "microwave is OPEN and OFF, microwave ON kitchencounter."
You have: "salmon."
assert('microwave' is 'opened')
LLM [GPT-3]
Correct Prediction
True
False
def microwave_salmon():
...
5: put salmon in microwave
...
assert('microwave' is 'opened')
else open('microwave')
putin('salmon', 'microwave')
...
Env [VH]
Microwave
Salmon
Full Execution
walk(kitchen)
find(salmon)
grab(salmon)
find (microwave)
open (microwave)
putin(salmon, microwave)
close (microwave)
switchon (microwave)
switchoff (microwave)
open (microwave)
grab(salmon)
close (microwave)


PROMPT for Planning
Import action primitives
from actions import walk <obj>, grab <obj>, switchon <obj>, open <obj>, standup, find <obj>, putin <obj> <obj>, ...
Available objects list
objects=[clothesshirt' sink, pie, apple, fridge, garbagecan, tv, dishwashingliquid, bed, bookshelf, salmon, stove, plate, coffeepot, kitchentable, wineglass, paper, microwave, toothbrush, toothpaste, bathroomcabinet, kitchen, lime, painkillers, barsoap, ...]
Example task(s)
def throw_away_lime():
# 0: find lime
find('lime')
...
# 5: close garbagecan
assert('close' to 'garbagecan')
else: find('garbagecan')
assert('garbagecan' is 'opened')
else: open('garbagecan')
close('garbagecan')
...
6: Done
Next task prompt
def microwave_salmon():
LLM [GPT-3]
Generated Plan
def microwave_salmon():
# 0: walk to kitchen
walk('kitchen')
# 1: find microwave
find('microwave')
# 2: open microwave
assert('close' to 'microwave')
else: find('microwave')
assert('microwave' is 'closed')
else: close('microwave')
open('microwave')
...
# 5: put salmon in microwave
assert('salmon' in 'hands')
else: find('salmon')
else: grab('salmon')
assert('close' to 'microwave')
else: find('microwave')
assert('microwave' is 'opened')
else: open('microwave')
putin('salmon', 'microwave')
...
close('microwave')
...
switchon('microwave')
# 8: wait for salmon to be done
cooking 1-2 minutes
9: turn off microwave
...
switchoff('microwave')
...
open('microwave')
...
grab('salmon')
# 12: put salmon on plate
assert('salmon' in 'hands')
...
assert('close' to 'plate')
else: find('plate')
putin('salmon', 'plate')
...
close('microwave')
# 14: Done
Comment: Summary
Action API + object(s): Plan Step
Comment - Action(s) Mapping
Assertions: State Feedback
Else: Recovery Actions
Optional Steps

PROMPT for Planning

Import action primitives

```
from actions import walk <obj>,
grab <obj>, switchon <obj>,
open <obj>, standup, find
<obj>, putin <obj> <obj>, ...
```

Available objects list

```
objects=[clothesshirt' sink,
pie, apple, fridge, garbagecan,
tv, dishwashingliquid, bed,
bookshelf, salmon, stove, plate,
coffeepot, kitchentable,
wineglass, paper, microwave,
toothbrush, toothpaste,
bathroomcabinet, kitchen, lime,
painkillers, barsoap, ...]
```

Example task(s)

```
def throw_away_lime():
  # 0: find lime
  find('lime')
  ...
  # 5: close garbagecan
  assert('close' to 'garbagecan')
     else: find('garbagecan')
  assert('garbagecan' is 'opened')
     else: open('garbagecan')
  close('garbagecan')
  ...
  # 6: Done
```

Next task prompt

```
def microwave_salmon():
```

LLM [GPT-3]

Generated Plan

```
def microwave_salmon():
 # 0: walk to kitchen
 walk('kitchen')
 # 1: find microwave
 find('microwave')
 # 2: open microwave
 assert('close' to 'microwave')
    else: find('microwave')
 assert('microwave' is 'closed')
    else: close('microwave')
 open('microwave')
 ...
 # 5: put salmon in microwave
 assert('salmon' in 'hands')
    else: find('salmon')
    else: grab('salmon')
 assert('close' to 'microwave')
    else: find('microwave')
 assert('microwave' is 'opened')
    else: open('microwave')
 putin('salmon', 'microwave')
 ...
 close('microwave')
 ...
 switchon('microwave')
 # 8: wait for salmon to be done
       cooking 1-2 minutes
 # 9: turn off microwave
 ...
 switchoff('microwave')
 ...
 open('microwave')
 ...
 grab('salmon')
 # 12: put salmon on plate
 assert('salmon' in 'hands' )
 ...
 assert('close' to 'plate' )
    else: find('plate')
 putin('salmon', 'plate')
 ...
 close('microwave')
 # 14: Done
```

Comment: Summary

Action API + object(s): Plan Step

Comment - Action(s) Mapping

Assertions: State Feedback

Else: Recovery Actions

Optio Steps

PROMPT for State Feedback

Example assertion check(s)

```
You see: "fridge is CLOSED,
lightswitch is ON, cereal,
bookshelf, box INSIDE bookshelf,
cereal ON wallshelf, paper
INSIDE bookshelf..."
You have: "book"

assert('close' to 'mug' )
False
assert('book' in 'hands')
True
assert('cereal' on 'bookshelf')
False
...
```

Current Semantic State

```
You see: "microwave is OPEN and
OFF, microwave ON
kitchencounter."
You have: "salmon."
```

```
assert('microwave' is 'opened')
```

LLM [GPT-3]

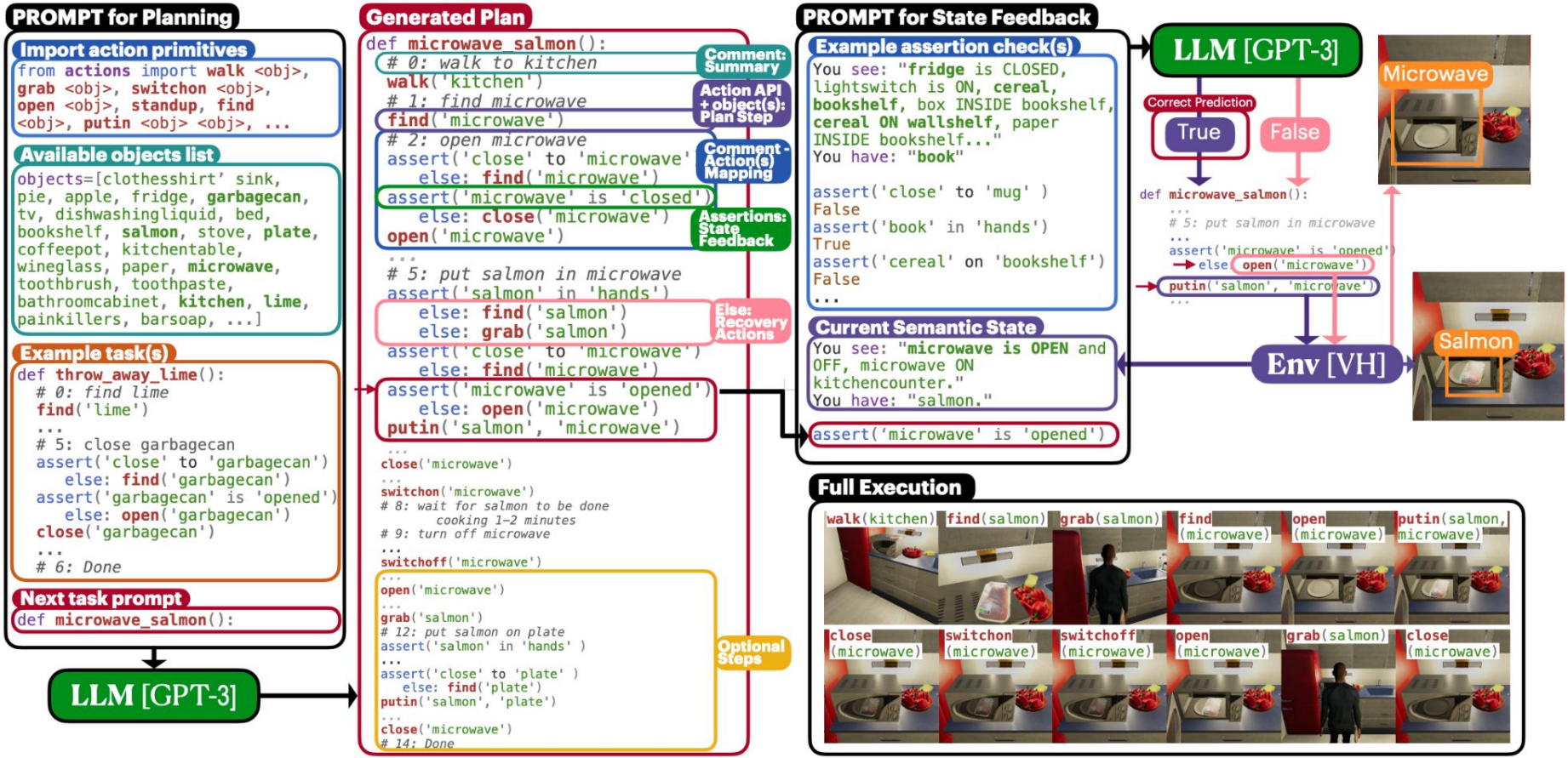
Correct Prediction

True

False

```
def microwave_salmon():
   ...
   # 5: put salmon in microwave
   ...
   assert('microwave' is 'opened')
      else open('microwave')
   putin('salmon', 'microwave')
   ...
```

Env [VH]


PROMPT for Planning
Import action primitives
from actions import walk <obj>, grab <obj>, switchon <obj>, open <obj>, standup, find <obj>, putin <obj> <obj>, ...
Available objects list
objects=[clothesshirt' sink, pie, apple, fridge, garbagecan, tv, dishwashingliquid, bed, bookshelf, salmon, stove, plate, coffeepot, kitchentable, wineglass, paper, microwave, toothbrush, toothpaste, bathroomcabinet, kitchen, lime, painkillers, barsoap, ...]
Example task(s)
def throw_away_lime():
# 0: find lime
find('lime')
...
# 5: close garbagecan
assert('close' to 'garbagecan')
else: find('garbagecan')
assert('garbagecan' is 'opened')
else: open('garbagecan')
close('garbagecan')
...
6: Done
Next task prompt
def microwave_salmon():
LLM [GPT-3]
Generated Plan
def microwave_salmon():
# 0: walk to kitchen
walk('kitchen')
# 1: find microwave
find('microwave')
# 2: open microwave
assert('close' to 'microwave')
else: find('microwave')
assert('microwave' is 'closed')
else: close('microwave')
open('microwave')
...
# 5: put salmon in microwave
assert('salmon' in 'hands')
else: find('salmon')
else: grab('salmon')
assert('close' to 'microwave')
else: find('microwave')
assert('microwave' is 'opened')
else: open('microwave')
putin('salmon', 'microwave')
...
close('microwave')
...
switchon('microwave')
# 8: wait for salmon to be done
cooking 1-2 minutes
9: turn off microwave
...
switchoff('microwave')
...
open('microwave')
...
grab('salmon')
# 12: put salmon on plate
assert('salmon' in 'hands')
...
assert('close' to 'plate')
else: find('plate')
putin('salmon', 'plate')
...
close('microwave')
# 14: Done
Comment: Summary
Action API + object(s): Plan Step
Comment - Action(s) Mapping
Assertions: State Feedback
Else: Recovery Actions
Optional Steps
PROMPT for State Feedback
Example assertion check(s)
You see: "fridge is CLOSED, lightswitch is ON, cereal, bookshelf, box INSIDE bookshelf, cereal ON wallshelf, paper INSIDE bookshelf..."
You have: "book"
assert('close' to 'mug')
False
assert('book' in 'hands')
True
assert('cereal' on 'bookshelf')
False
...
Current Semantic State
You see: "microwave is OPEN and OFF, microwave ON kitchencounter."
You have: "salmon."
assert('microwave' is 'opened')
LLM [GPT-3]
Correct Prediction
True
False
def microwave_salmon():
...
5: put salmon in microwave
...
assert('microwave' is 'opened')
else open('microwave')
putin('salmon', 'microwave')
...
Env [VH]
Microwave
Salmon
Full Execution
walk(kitchen)
find(salmon)
grab(salmon)
find (microwave)
open (microwave)
putin(salmon, microwave)
close (microwave)
switchon (microwave)
switchoff (microwave)
open (microwave)
grab(salmon)
close (microwave)

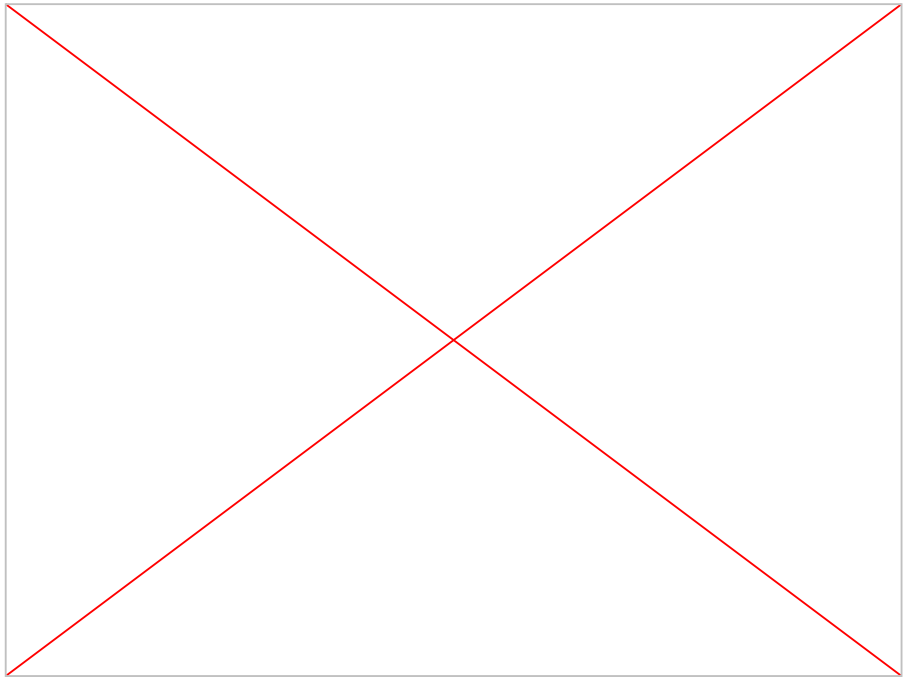
Experiment - Evaluation Metrics

- Success Rate (SR): the fraction of executions that achieved all task-relevant goal-conditions
- Goal Conditions Recall (GCR): the percentage of critical goals that were actually accomplished
- Executability (Exec): the fraction of actions in the generated plan that are actually executable by the robot in the environment

Experiment - Simulation Experiments

Virtual Home (VH) Environment

- A VH state s is a set of objects O and properties P.
- P encodes information like in(salmon, microwave) and agent close to(salmon).
- VH provides observations in the form of state graph with object properties and relations



Example task: put salmon in the fridge

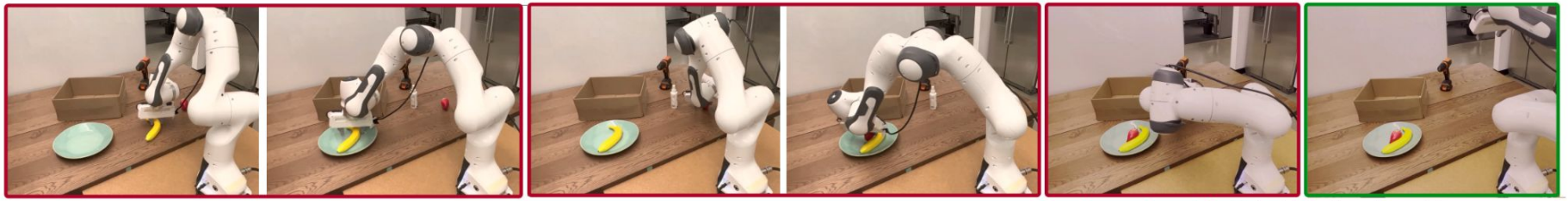
#	— Prompt Format and Parameters — Format	COMMENTS	FEEDBACK	LLM Backbone	*SR*	*Exec*	*GCR*
1	PROGPROMPT	✓	✓	CODEX	**0.40**±0.11	**0.90**±0.05	**0.72**±0.09
2	PROGPROMPT	✓	✓	DAVINCI	0.22±0.04	0.60±0.04	0.46±0.04
3	PROGPROMPT	✓	✓	GPT3	0.34±0.08	0.84±0.01	0.65±0.05
4	PROGPROMPT	✓	✗	GPT3	0.28±0.04	0.82±0.01	0.56±0.02
5	PROGPROMPT	✗	✓	GPT3	0.30±0.00	0.65±0.01	0.58±0.02
6	PROGPROMPT	✗	✗	GPT3	0.18±0.04	0.68±0.01	0.42±0.02
7	LANGPROMPT	-	-	GPT3	0.00±0.00	0.36±0.00	0.42±0.02
8	Baseline from HUANG ET AL. [2]			GPT3	0.00±0.00	0.45±0.03	0.21±0.03

Observation:

- PROGPROMPT outperforms prior work
- CODEX exceeds GPT3 performance
- FEEDBACK improve performance
- Removing COMMENTS from the prompt code substantially reduces performance

Experiment - Real-Robot Experiments

Task: sort fruits on the plate and bottles in the box



`grab_and_puton('banana', 'plate')` `grab_and_puton('strawberry', 'plate')` `grab_and_putin('bottle', 'box')`

- No feedback mechanism: Physical robot setup did not allow reliably tracking system state and checking assertions, and is prone to random failures due to things like grasps slipping.
- Introduce an additional metric:
- Plan SR: whether the plan would have likely succeeded, provided successful pick-and-place execution without gripper failures.

Task Description	Distractors	*SR*	Plan *SR*	*GCR*
put the banana in the bowl	0	1	1	1/1
	4	1	1	1/1
put the pear on the plate	0	1	1	1/1
	4	1	1	1/1
put the banana on the plate and the pear in the bowl	0	1	1	2/2
	3	1	1	2/2
sort the fruits on the plate and the bottles in the box	0	0	1	2/3
	1	1	1	3/3
	2	0	0	2/3

Observation:

- Across tasks, with and without distractor objects, the system almost always succeeds, failing only on the sort task.

Strengths and Weaknesses

✅ Long-Horizon Planning
✅ Generalization to New Tasks and Environments
✅ Executable and Valid Task Plans
✅ Error Recovery via Precondition Checking (Feedback)

❌ Environment interaction during execution is limited to the commonsense precondition checking-based feedback
❌ Focus on Semantic State Only
❌ Limited Information Exchange (Text-Only Modality)

Questions?